Python Input & Output

Communicating with your programs — getting data in, sending results out, and making it all readable.

Made with GAMMA

The Foundation: Getting Data In and Out

Why It Matters

Every useful program communicates. Without input and output, your code runs in silence — no questions asked, no answers given. I/O is the bridge between your logic and the real world.

The Two Core Tools

- `print()` — sends information **out** to the user
- `input()` — receives information **in** from the user

Master these two functions and you have the foundation for interactive, dynamic Python programs.

Made with GAMMA

Displaying Output: The `print()` Function

Your primary tool for showing information — strings, numbers, variables, and more.

Basic String Output

```
print("Hello, Python!")
```

Outputs: `Hello, Python!`

Printing Variables

```
name = "Ada"
print(name)
```

Outputs: `Ada`

Mixing Types

```
age = 30
print("Age:", age)
```

Outputs: `Age: 30`

Made with GAMMA

Receiving Input: The `input()` Function

`input()` pauses your program and waits for the user to type something. It always returns the response as a **string**.

```
name = input("Enter your name: ")
print("Hello,", name)
```

The prompt string inside `input()` is displayed to the user before they type — make it clear and descriptive.

Made with GAMMA

Beyond Basic Output: Fancier Formatting

Python offers powerful ways to build polished, readable output strings.

f-Strings (Recommended)

The modern, most readable approach. Embed variables directly inside curly braces.

```
year = 2024
print(f"The year is {year}")
```

Outputs: `The year is 2024`

str.format() Method

A powerful older approach — great for precise number and alignment formatting.

```
"{:} YES votes {:2.2%}".format(
 yes_votes, percentage)
```

Ideal for controlling decimal places and column widths.

Made with GAMMA

Controlling `print()`'s Behavior

Two optional arguments give you fine-grained control over how output is displayed.

The `sep` Argument

Changes the separator inserted **between** multiple items. Default is a space.

```
print("a", "b", "c", sep="...")
# Output: a...b...c
```

```
print("2024", "01", "15", sep="-")
# Output: 2024-01-15
```

The `end` Argument

Changes what's printed at the **end** of the call. Default is a newline `\n`.

```
print("Hello", end=" ")
print("World!")
# Output: Hello World!
```

Set `end=""` to keep multiple prints on the same line.

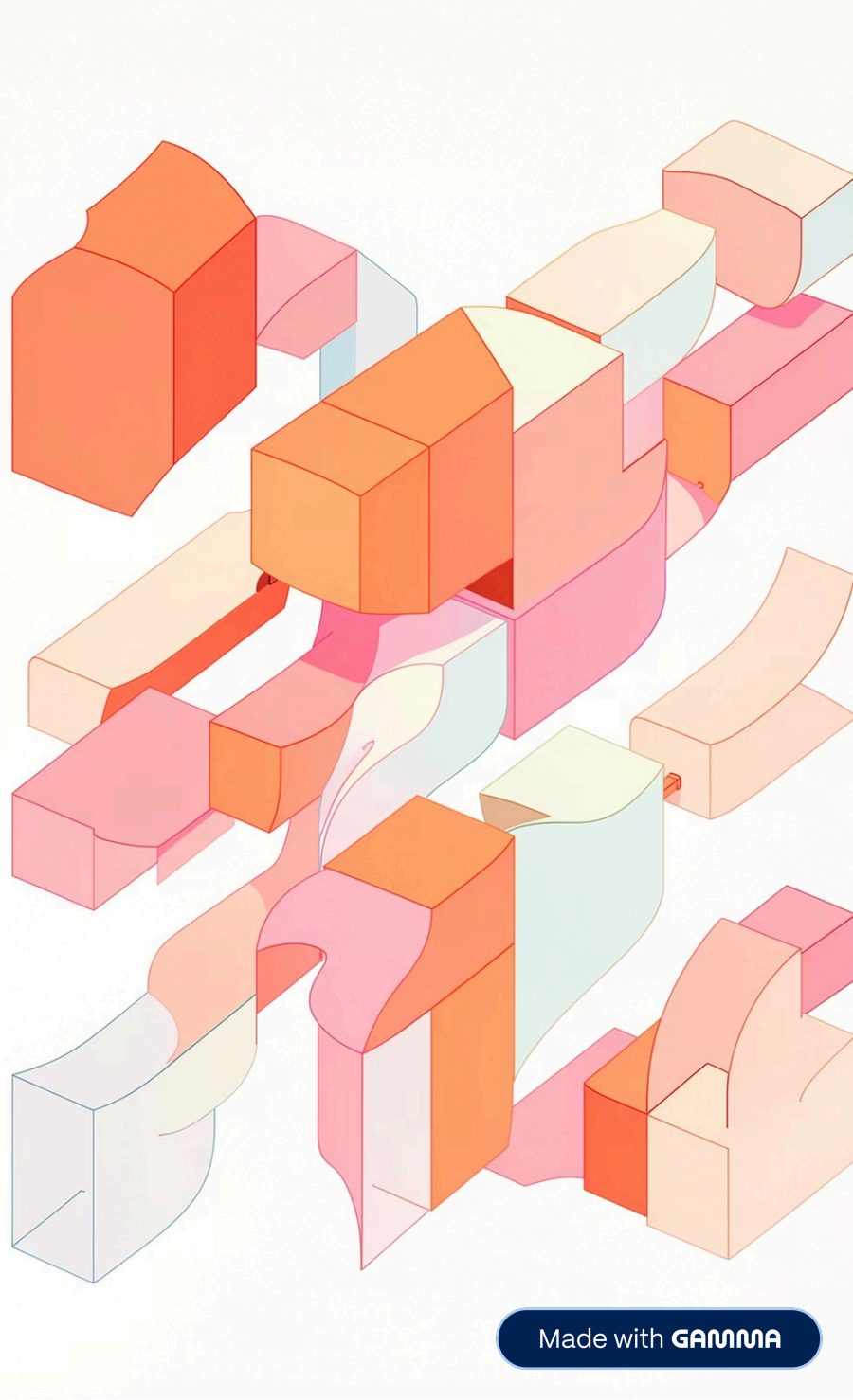
Made with GAMMA

Data Types and Input

`input()` **always** returns a string — even if the user types a number. You must explicitly convert it.

1. **User Types**
   `"25"` — a string
2. **You Convert**
   `int(input(...))`
3. **You Get**
   `25` — an integer

⚠ Forgetting to convert input types is one of the most common Python beginner errors. Always check what type you need before performing calculations.



Made with GAMMA

str() vs. repr(): Different Views of Data

Two Ways to Represent Data

Python provides two built-in functions for converting objects to strings — each with a distinct purpose.

- str() — human-friendly, meant for display
- repr() — developer-friendly, unambiguous, often valid Python syntax

A good rule: use str() for end users, repr() for debugging.

Side-by-Side Example

```
text = "Hello"

str(text)
# 'Hello'

repr(text)
# "'Hello'"
```

```
import datetime
d = datetime.date.today()

str(d) # '2024-01-15'
repr(d) # 'datetime.date(2024, 1, 15)'
```

Made with GAMMA

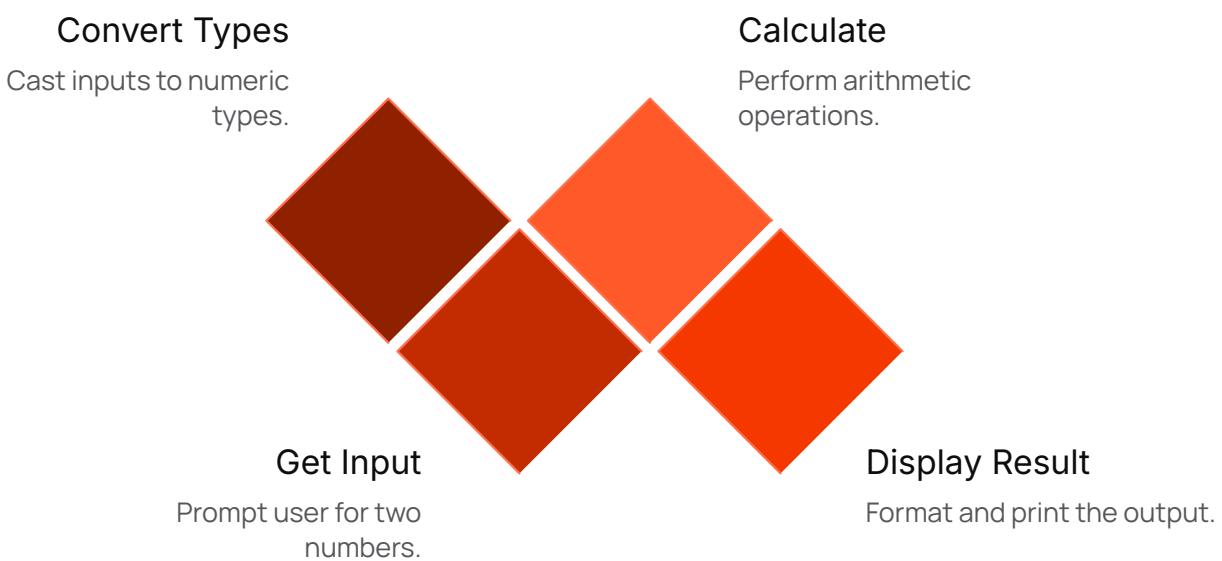
Practical Example: A Simple Calculator

Putting it all together — input, type conversion, calculation, and formatted output.

```
# Get numbers from the user
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))

# Perform calculation
result = a + b

# Display formatted result
print(f"{a} + {b} = {result}")
```



This simple pattern — input, convert, compute, display — is the skeleton of countless real-world Python programs.

Made with GAMMA

Key Takeaways: Mastering Python I/O

print() for Output

Your go-to for displaying any information to the user.

input() for Input

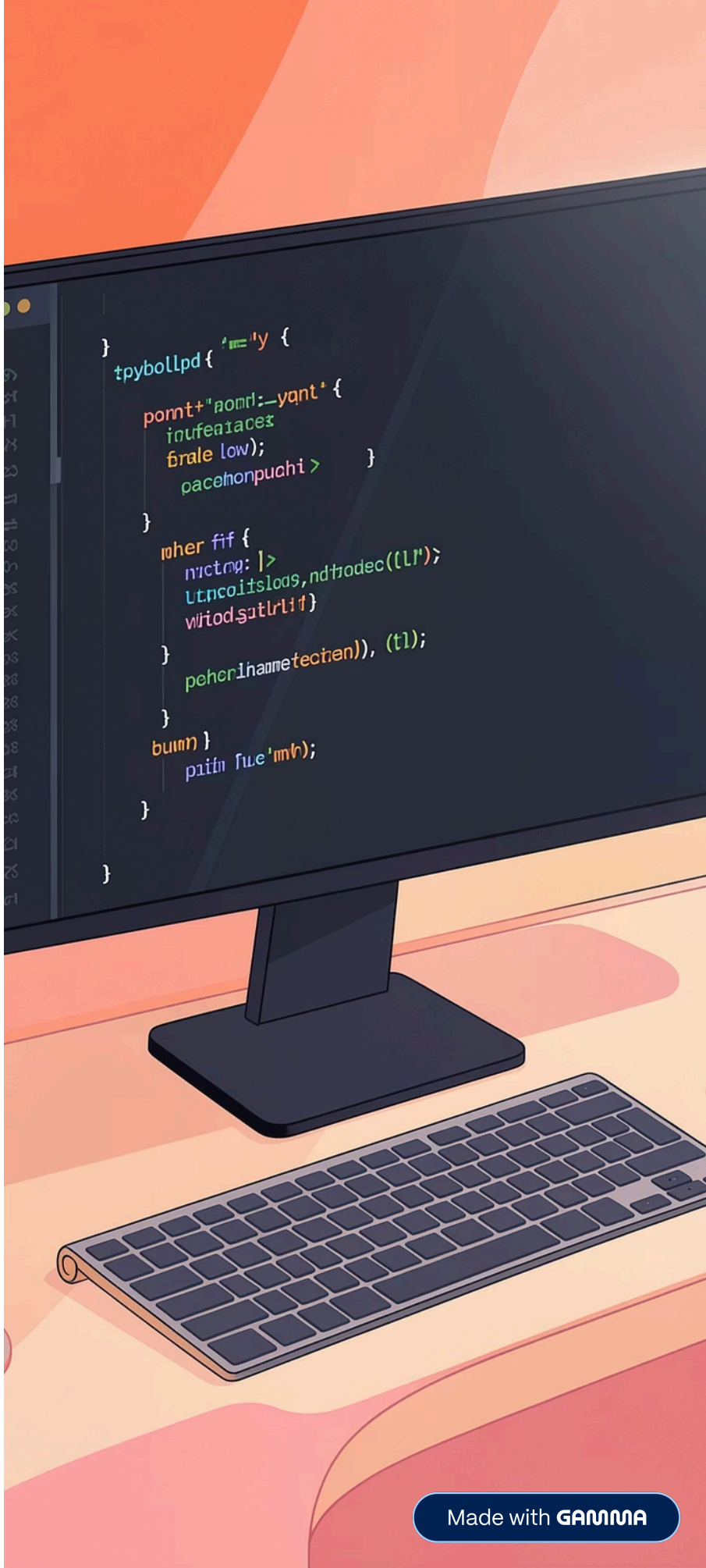
Pauses and collects user responses — always as a string.

Use f-Strings

Clear, concise, and readable — the modern formatting standard.

Convert Your Types

Always cast input to the right type before using it in logic.



Made with GAMMA